

GEVK HTIService API Programming User Guide

Version: 0.0.2

Release Notes.....	3
HTI API	4
1.1. Configuration	4
1.1.1. Initialization	4
1.1.2. Service Version	5
1.1.3. API Version.....	5
1.1.4. Callback Configuration.....	5
1.1.5. Miscellaneous Configuration	7
1.2. Intent.....	21
1.3. Sample Code	22

Release Notes

Version	Date	Notes
0.0.2	2025/10/15	▶ 1.1.4. Callback Configuration – ▶ Modify SerialPort_ReadData SerialPort_ReadData_String ▶ 1.1.5. Miscellaneous Configuration – ▶ Modify SerialPort_Close SerialPort_WriteData
0.0.1	2025/10/02	First Release

HTI API

Before developing your self-made application, the offered "**htiapi_v0_x_xx.jar**" library file has to be imported into your project.

Library Required	Location
htiapi_v0_x_xx.jar (for Android Studio or Eclipse)	/storage/emulated/0/Documents/HTI

1.1. Configuration

1.1.1. Initialization

InitInstance

Purpose	Creates a HTIManager instance and sets the remote callback function (See 1.1.4)before employing any APIs.
Syntax	HTIManager InitInstance(Context context ,RemoteHTICallBack remoteCallBack);
Example	<pre>private HTIManager mHTIManager; mHTIManager = HTIManager.InitInstance(this, mCallBack);</pre>
Return Value	Gets a HTIManager instance if successful, else null.

Release

Purpose	Releases resources.
Syntax	void Release ();
Example	mHTIManager.Release();

1.1.2. Service Version

GetServiceVersion

Purpose	Obtains the HTI service version.
Syntax	String GetServiceVersion ()
Example	<code>String ver = mHTIManager.GetServiceVersion();</code>

1.1.3. API Version

GetAPIVersion

Purpose	Obtains the HTI service api version.
Syntax	String GetAPIVersion ()
Example	<code>String ver = mHTIManager.GetAPIVersion();</code>

1.1.4. Callback Configuration

Parameters

RemoteHTICallBack *remoteCallBack*

statusCallBackVisible()	The callback is enabled
statusCallBackInvisible()	The callback is disabled
sendMessage(String s)	Get the message
CAN_ReadData(long[] data)	Get the CAN Bus read bytes data. The data length is 12. 0:canid; 1:eff; 2:rtr; 3:len; 4~11:data.
SerialPort_ReadData(String path ,byte[] data)	Get the Serial Port read path and bytes data
SerialPort_ReadData_String(String path ,String data)	Get the Serial Port read path and string data
GPIO_IRQ_ReadData(String index)	Get the GPIO IRQ pin index (0-9).

Example

```
RemoteHTICallBack mCallBack = new RemoteHTICallBack() {

    @Override

    public void statusCallBackVisible() {Log.d(TAG,"statusCallBackVisible ");}

    @Override

    public void statusCallBackInvisible() {Log.d(TAG,"statusCallBackInvisible ");}

    @Override

    public void sendMessage(String s) {Log.d(TAG,"sendMessage = " + s);}

    @Override

    public void CAN_ReadData(long[] receive_data) {

        String re = ByteLongArrayToString(receive_data);

        Log.d(TAG," CAN_ReadData = " + re );}

    @Override

    public void SerialPort_ReadData(String path ,byte[] s) {

        String data = toHexString(s,s.length);

        Log.d(TAG," SerialPort_ReadData =" + data );}

    @Override

    public void SerialPort_ReadData(String path ,String s) {

        Log.d(TAG," SerialPort_ReadData =" + s );}

    @Override

    public void GPIO_IRQ_ReadData(String s) {

        Log.i(TAG,"[GPIO_IRQ_ReadData] = " + s) ;}

    };

    mHTIManager = HTIManager.InitInstance(this,mCallBack);
```

1.1.5. Miscellaneous Configuration

setI2C

Purpose Sets the I2C value.

Syntax **int setI2C(int port,int slave_address, int[] data);**

Parameters	<i>int port</i> The I2C bus port number. <i>int slave_address</i> The I2C slave device address. <i>int[] data</i> The data array to be transmitted to the I2C slave. The first element must be the register address inside the slave device. The subsequent elements are the data bytes to be written into that register.
Example	<pre>int re = mHTIManager.setI2C(5,0x38 , new int[]{0xAC, 0x33, 0x00}); if(re!=HTIResult.S_OK.getValue()) {String err = mHTIManager.GetLastError(); Log.e(TAG, "I2C_Write error = " + err); }</pre>
Return Value	If successful, it returns HTIResult.S_OK.ordinal(). Otherwise, it returns HTIResult.S_ERR.Ordinal().
See Also	getI2C

getI2C

Purpose	Reads data from the specified I2C slave device.
Syntax	int getI2C(int port,int slave_address, int length , int [] return_data);
Parameters	<i>int port</i> The I2C bus port number. <i>int slave_address</i> The I2C slave device address. <i>int length</i> The number of bytes to read from the I2C slave. <i>int[] data</i> The buffer array to store the data read from the I2C slave

Example	<pre> int [] return_value = new int[6]; int re = mHTIManager.getI2C(5,0x38 , 6,return_value); if(re!=HTIResult.S_OK.getValue()) {String err = mHTIManager.GetLastError(); Log.e(TAG, "I2C_Read error = " + err);} else{ StringBuilder sb = new StringBuilder(); for (int v : return_value) { sb.append(String.format("0x%02X ", v)); } Log.i(TAG, "[I2C_Read] = " + sb.toString().trim());} </pre>
Return Value	If successful, it returns HTIResult.S_OK.ordinal(). Otherwise, it returns HTIResult.S_ERR.Ordinal().
See Also	setI2C

setGPIO	
Purpose	Configures and sets the specified GPIO pin.
Syntax	int setGPIO(int pin_num, GPIO_Config option);

Parameters

A default value comes with an asterisk "*".

int *pin_num*

The target GPIO pin number (0-9).

GPIO_Config *option*

The configuration object that defines the GPIO settings, including direction, interrupt mode, potential, and trigger type.

GPIO_Direction *direction*

Defines the pin direction.

GPIO_Direction.input	* 0; Configure as input pin
GPIO_Direction.output	1; Configure as output pin

Boolean *interrupt*

Enables or disables interrupt function for the pin.

GPIO_Potential *potential*

Defines the default logic level of the pin.

GPIO_Potential.low	* 0; Low level
GPIO_Potential.high	1; High level
GPIO_Potential.open_drain	2; Open-drain output mode

GPIO_WayToEvok *wayToEvok*

Defines the interrupt trigger condition.

GPIO_WayToEvok.IRQF_TRIGGER_NONE	* 0; No trigger
GPIO_WayToEvok.IRQF_TRIGGER_RISING	1; Trigger on rising edge
GPIO_WayToEvok.IRQF_TRIGGER_FALLING	2; Trigger on falling edge
GPIO_WayToEvok.IRQF_TRIGGER_RISING_ and_IRQF_TRIGGER_FALLING	3; Trigger on both edges
GPIO_WayToEvok.IRQF_TRIGGER_HIGH	4; Trigger when pin is high
GPIO_WayToEvok.IRQF_TRIGGER_LOW	8; Trigger when pin is low

Example	<pre>// no IRQ low GPIO_Config config_low = new GPIO_Config(); config_low.direction = GPIO_Direction.output; config_low.interrupt = false; config_low.potential = GPIO_Potential.low; config_low.wayToEvok = GPIO_WayToEvok.IRQF_TRIGGER_NONE; // IRQ RISING GPIO_Config config_RISING = new GPIO_Config(); config_RISING.direction = GPIO_Direction.intput; config_RISING.interrupt = true; config_RISING.potential = GPIO_Potential.high; config_RISING.wayToEvok = GPIO_WayToEvok.IRQF_TRIGGER_RISING; // IRQ RISING and FALLING GPIO_Config config_irq= new GPIO_Config(); config_irq.direction = GPIO_Direction.intput; config_irq.interrupt = true; config_irq.potential = GPIO_Potential.high; config_irq.wayToEvok = GPIO_WayToEvok.IRQF_TRIGGER_RISING_and_IRQF_TRIGGER_FALLING; int re = mHTIManager.setGPIO(1, config_low); if(re!=HTIResult.S_OK.getValue()) {String err = mHTIManager.GetLastError(); Log.e(TAG, "new_GPIO_Write error = " + err);} </pre>
Return Value	If successful, it returns HTIResult.S_OK.ordinal(). Otherwise, it returns HTIResult.S_ERR.Ordinal().
See Also	getGPIO

getGPIO

Purpose Reads the current states of all GPIO pins.

Syntax `int getGPIO(boolean[] gpio);`

Parameters `Boolean[] gpio`

An array to store the state of each GPIO pin.

true	High level
false	Low level

Example

```
boolean [] value = new boolean[10];

int re = mHTIManager.getGPIO(value);

if(re!=HTIResult.S_OK.getValue())

{String err = mHTIManager.GetLastError();

Log.e(TAG, "new_GPIO_Read error = " + err); }

else

{StringBuilder sb = new StringBuilder();

sb = new StringBuilder();

for (boolean b : value) {

sb.append(b ? "1 " : "0 "); // true -> 1, false -> 0}

String hexString = sb.toString().trim();

Log.i(TAG, "gpio value: " + hexString); }
```

Return Value If successful, it returns HTIResult.S_OK.ordinal().
Otherwise, it returns HTIResult.S_ERR.Ordinal().

See Also `setGPIO`

SPI_Transfer

Purpose Transfers data to/from the specified SPI device.

Syntax `int SPI_Transfer(int index, byte[] data, byte[] out_data);`

Parameters	<code>int index</code> The SPI device index or channel number. (1-2) <code>byte[] data</code> The data array to be sent to the SPI device. <code>byte[] out_data</code> The buffer array to store data received from the SPI device.
Example	<pre>byte[] data={(byte)0x01, (byte) 0xaa}; byte [] out = new byte[data.length]; int re = mHTIManager.SPI_Transfer(1,data,out); if(re!=HTIResult.S_OK.getValue()) {String err = mHTIManager.GetLastError(); Log.e(TAG, "SPI_Transfer error = " + err); } else {String _out = toHexString(out,out.length); Log.i(TAG, "SPI_Transfer : " + _out); }</pre>
Return Value	If successful, it returns <code>HTIResult.S_OK.ordinal()</code>. Otherwise, it returns <code>HTIResult.S_ERR.Ordinal()</code>.

setPWM

Purpose	Configures and enables/disables the specified PWM channel.
Syntax	<code>int setPWM(int index, int enable, int period, int duty_cycle);</code>

Parameters *int index*

The PWM channel index.(1-2)

int enable

Enable or disable the PWM output.

1	Enable PWM output
0	Disable PWM output

int period

The PWM frequency in Hertz (Hz). (100Hz~10000Hz)

int duty_cycle

The PWM duty cycle as a percentage of the period. (0~100%)

Example

```
int re = mHTIManager.setPWM(1, 1, 100, 50);

if(re!=HTIResult.S_OK.getValue())

{String err = mHTIManager.GetLastError();

Log.e(TAG, "PWM error = " + err); }
```

Return Value

If successful, it returns HTIResult.S_OK.ordinal().

Otherwise, it returns HTIResult.S_ERR.Ordinal().

CAN_Open

Purpose Open the CAN Bus.

Syntax **int CAN_Open(String name)**

Parameters String name

The CAN Bus name.

Example

```
int re = mHTIManager.CAN_Open("can0");

if(re!=HTIResult.S_OK.getValue())

{String err = mHTIManager.GetLastError();

Log.e(TAG, "error = " + err);}
```

Return Value

If successful, it returns HTIResult.S_OK.ordinal().

Otherwise, it returns HTIResult.S_ERR.Ordinal().

See Also

CAN_Close 、 CAN_Write 、 CAN_ReadData

CAN_Close

Purpose	Close the CAN Bus.
Syntax	int CAN_Close()
Example	<pre>int re = mHTIManager.CAN_Close(); if(re!=HTIResult.S_OK.getValue()) {String err = mHTIManager.GetLastError(); Log.e(TAG, "CAN_Close error = " + err);} </pre>
Return Value	If successful, it returns HTIResult.S_OK.ordinal(). Otherwise, it returns HTIResult.S_ERR.Ordinal().
See Also	CAN_Open 、CAN_Write 、CAN_ReadData

CAN_Write

Purpose	Write data to the CAN Bus.
Syntax	int CAN_Write(long canid, long eff,long rtr,int len, in int[] data)
Parameters	Long <i>canid</i> Can id Long <i>eff</i> Extended CAN frame is 1, Standard CAN Frame is 0; Long <i>rtr</i> Remote Transmit Request. Default is 0. When RTR=0, it means sending data out, and RTR=1 means asking the remote party to send data back. Int <i>len</i> The data length. Int[] <i>data</i> The write data.

Example

```
int[] data = {0xA0, 0xA1, 0xA2, 0xA3, 0xA4, 0xA5, 0xA6, 0xA7};

int re = mHTIManager.CAN_Write(0x12345678,1,0, data.length,data);

//re = mHTIManager.CAN_Write(0x123,0,0, data.length,data);

if(re!=HTIResult.S_OK.getValue())

{String err = mHTIManager.GetLastError();

Log.e(TAG, "CAN_Write error = " + err);}
```

Return Value If successful, it returns HTIResult.S_OK.ordinal().
Otherwise, it returns HTIResult.S_ERR.Ordinal().

See Also CAN_Open 、CAN_Close 、CAN_ReadData

SerialPort_Open

Purpose Open the Serial Port.

Syntax **int SerialPort_Open(String path ,UART_baudRate baudRate)**

Parameters A default value comes with an asterisk "*".

String *path*

The UART path. (dev/ttyS1 and dev/ttyS2)

UART_baudRate *baudRate*

UART_baudRate.BaudRate_300	Baud Rate 300
UART_baudRate.BaudRate_600	Baud Rate 600
UART_baudRate.BaudRate_1200	Baud Rate 1200
UART_baudRate.BaudRate_2400	Baud Rate 2400
UART_baudRate.BaudRate_4800	Baud Rate 4800
UART_baudRate.BaudRate_9600	* Baud Rate = 9600
UART_baudRate.BaudRate_19200	Baud Rate = 19200
UART_baudRate.BaudRate_38400	Baud Rate = 38400
UART_baudRate.BaudRate_57600	Baud Rate = 57600
UART_baudRate.BaudRate_115200	Baud Rate = 115200
UART_baudRate.BaudRate_230400	Baud Rate = 230400
UART_baudRate.BaudRate_460800	Baud Rate = 460800
UART_baudRate.BaudRate_961600	Baud Rate = 961600

Example

```
int re =
mHTIManager.SerialPort_Open("dev/ttyS1",UART_baudRate.BaudRate_9600);

if(re!=HTIResult.S_OK.getValue())

{String err = mHTIManager.GetLastError();

Log.e(TAG, "SerialPort_Open error = " + err);}
```

Return Value If successful, it returns HTIResult.S_OK.ordinal().
Otherwise, it returns HTIResult.S_ERR.Ordinal().

See Also SerialPort_Close 、 SerialPort_WriteData 、 SerialPort_ReadData

SerialPort_Close

Purpose Close the Seral Port.

Parameters	String <i>path</i> The UART path. (dev/ttyS1 and dev/ttyS2)
Syntax	int SerialPort_Close(String path)
Example	<pre>int re = mHTIManager.SerialPort_Close("dev/ttyS1"); if(re!=HTIResult.S_OK.getValue()) {String err = mHTIManager.GetLastError(); Log.e(TAG, "error = " + err);}</pre>
Return Value	If successful, it returns HTIResult.S_OK.ordinal(). Otherwise, it returns HTIResult.S_ERR.Ordinal().
See Also	SerialPort_Open 、 SerialPort_WriteData 、 SerialPort_ReadData

SerialPort_WriteData

Purpose	Write data to the Serial Port
Syntax	int SerialPort_WriteData(String path ,byte[] data) int SerialPort_WriteData(String path ,String data)
Parameters	String <i>path</i> The UART path. (dev/ttyS1 and dev/ttyS2) byte[] <i>data</i> ; String <i>data</i> The write data.

Example	<pre>String msg = "1234"; byte[] temp = toByteArray(msg); int re = mHTIManager.SerialPort_WriteData("dev/ttyS1",temp); if(re!=HTIResult.S_OK.getValue()) {String err = mHTIManager.GetLastError(); Log.e(TAG, "error = " + err);} //////////////////////////////////// String msg = "1234"; int re = mHTIManager.SerialPort_WriteData("dev/ttyS1",msg); if(re!=HTIResult.S_OK.getValue()) {String err = mHTIManager.GetLastError(); Log.e(TAG, "error = " + err);}</pre>
Return Value	If successful, it returns HTIResult.S_OK.ordinal(). Otherwise, it returns HTIResult.S_ERR.Ordinal().
See Also	SerialPort_Open 、 SerialPort_Close 、 SerialPort_ReadData

SuspendDevice

Purpose	Suspend device.
Syntax	void SuspendDevice()
Example	mHTIManager.SuspendDevice();

ResumeDevice

Purpose	Resume device.
Syntax	void ResumeDevice()
Example	mHTIManager.ResumeDevice();

SetSystemTime

Purpose	Sets system time.
---------	-------------------

Syntax	void SetSystemTime(in SystemTime time);
Example	<pre>SimpleDateFormat sdf = new SimpleDateFormat("yyyy/MM/dd HH:mm:ss"); Date dt = null; try {dt = sdf.parse("2020/12/31 11:49:00");} catch (ParseException e) {e.printStackTrace();} SystemTime msystemtime = new SystemTime(); msystemtime.time = dt; mHTIManager.SetSystemTime(msystemtime);</pre>

SetAutoSyncTime

Purpose	Sets the network-provided time. (Settings > System > Date & time > Use network-provided time)
Syntax	void SetAutoSyncTime(boolean status);
Example	<code>mHTIManager.SetAutoSyncTime(true);</code>

GetAutoSyncTime

Purpose	Gets the network-provided time. (Settings > System > Date & time > Use network-provided time)
Syntax	boolean GetAutoSyncTime();
Example	<code>boolean autotime = mHTIManager.GetAutoSyncTime();</code>
Return Value	If the network-provided time is enabled, it returns true. Otherwise, it returns false.

RebootDevice

Purpose	Reboot device.
Syntax	void RebootDevice();
Example	<code>mHTIManager.RebootDevice();</code>

1.2. Intent

GeneralString.Intent_HTISERVICE_CONNECTED

Purpose After running InitInstance, the system makes connection between the application and the HTI service. With success in making connection, this intent is sent.

1.3. Error Code Reference

Error Code	Message
0	OK
1	Parameter is incorrect
999	The API does not support this project
1012	Failed to open CAN
1013	Failed to close CAN
1014	CAN write data failed
1015	CAN needs to be initialized first. (CAN_Open)
1016	Serial port open failed
1017	Serial port close failed
1018	Serial port write data failed
1019	Serial port write data failed, The serial port is not open.
1024	SPI write data failed

Error Code	Message
1025	PWM write data failed. Invalid period range. (100Hz - 10000Hz)
1026	PWM write data failed. Invalid duty cycle range. (0 - 100)
1027	PWM write data failed. The duty cycle must not be greater than the period.
1028	I2C write data failed.
1029	I2C read data failed.
1030	GPIO write data failed.
1031	GPIO read data failed. The provided GPIO array is too small to hold the returned values.
1032	GPIO read data failed.

1.4. Sample Code

```
package com.example.myapplication;

import androidx.appcompat.app.AppCompatActivity;
import android.app.Activity;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.content.IntentFilter;
import android.os.Bundle;
```

```
import android.util.Log;

import android.view.View;

import android.widget.AdapterView;

import android.widget.AdapterView.OnItemClickListener;

import android.widget.AdapterView.OnItemSelectedListener;

import android.widget.CheckBox;

import android.widget.CompoundButton;

import android.widget.EditText;

import android.widget.RadioButton;

import android.widget.RadioGroup;

import android.widget.Spinner;

import android.widget.Switch;

import android.widget.TextView;

import android.widget.Toast;

import com.hti.callback.RemoteHTICallBack;

import com.hti.htiapi.HTIManager;

import com.hti.util.CANBUS_BaudRate;

import com.hti.util.GPIO_Config;

import com.hti.util.GPIO_Direction;

import com.hti.util.GPIO_Potential;

import com.hti.util.GPIO_WayToEvok;

import com.hti.util.GeneralString;

import com.hti.util.HTIResult;

import com.hti.util.SystemTime;

import com.hti.util.UART_baudRate;

import java.text.ParseException;
```

```
import java.text.SimpleDateFormat;

import java.util.Date;

public class MainActivity extends Activity {

    String TAG = "HTI_sample";

    Context mContext;

    HTIManager mHTIManager = null;

    private TextView mTextView ;

    public Switch m_switch = null;

    private boolean IsCallBack = false;

    private Spinner spinnerBaudRate ,spinnerSPI ,spinnerGPIO,spinner_pwm;

    private RadioGroup radioGroupGPIO ;

    private EditText editText_test ;

    private CheckBox checkBox_pwm;

    private EditText editText_pwm_period;

    private EditText editText_pwm_duty;

    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        mContext = MainActivity.this;

        RemoteHTICallback mCallBack = new RemoteHTICallback() {

            @Override

            public void statusCallBackVisible() {

                IsCallBack = true;

            }

        }

    }

}
```

```
@Override

public void statusCallBackInvisible() {

    IsCallBack = false;

}

@Override

public void sendMessage(String s) {

}

@Override

public void CAN_ReadData(long[] longs) {

    String re = ByteLongArrayToString(longs);

    Log.d(TAG," CAN_ReadData  = "  + re );

}

@Override

public void SerialPort_ReadData(String path,byte[] bytes) {

    String data = toHexString(bytes,bytes.length);

    Log.d(TAG," SerialPort_ReadData  ="  + data );

}

@Override

public void SerialPort_ReadData(String path ,String s) {

    Log.d(TAG," SerialPort_ReadData = "  + s );

}

@Override

public void GPIO_IRQ_ReadData(String s) {

    Log.i(TAG,"GPIO_IRQ_ReadData = " + s );

}
```

```
};

mHTIManager = HTIManager.InitInstance(this,mCallBack);

IntentFilter filter = new IntentFilter();

filter.addAction(GenericString.Intent_HTISERVICE_CONNECTED);

registerReceiver(myDataReceiver, filter);

ui_init();

}

@Override

protected void onDestroy() {

    super.onDestroy();

    unregisterReceiver(myDataReceiver);

    Log.i(TAG,"onDestroy");

    if(mHTIManager!=null)

    {

        mHTIManager.Release();

    }

}

private final BroadcastReceiver myDataReceiver = new BroadcastReceiver()

{

    @Override

    public void onReceive(Context context, Intent intent) {

        if

(intent.getAction().equals(GenericString.Intent_HTISERVICE_CONNECTED))

        {
```

```

String PackageName = intent.getStringExtra("PackageName");

// make sure this AP does already connect with HTI service (after
call HTIManager.InitInstance(this)

String ver = mHTIManager.GetServiceVersion();

String api_ver = mHTIManager.GetAPIVersion();

Toast.makeText(MainActivity.this, PackageName,
Toast.LENGTH_SHORT).show();

Log.i(TAG,PackageName);

Log.i(TAG,"ServiceVersion = " + ver);

Log.i(TAG,"APIVersion = " + api_ver);

    }

}

};

public static String toHexString(byte[] arg, int length) {

    String result = new String();

    if (arg != null) {

        for (int i = 0; i < length; i++) {

            result = result

                + (Integer.toHexString(

                    arg[i] < 0 ? arg[i] + 256 : arg[i]).length() == 1 ? "0"

                    + Integer.toHexString(arg[i] < 0 ? arg[i] + 256

                        : arg[i])

                    : Integer.toHexString(arg[i] < 0 ? arg[i] + 256

                        : arg[i])) + " ";

        }

    }

```

```
        return result;
    }

    return "";
}

private String ByteLongArrayToString(long[] longs) {
    if (longs == null || longs.length == 0) {
        return "";
    }

    StringBuilder sb = new StringBuilder();
    for (long value : longs) {
        sb.append(String.format("%016X", value)).append(" ");
    }

    return sb.toString().trim();
}

public void SET_Systemtime(View view)
{
    SimpleDateFormat sdf = new SimpleDateFormat("yyyy/MM/dd HH:mm:ss");
    Date dt = null;
    try {
        dt = sdf.parse("2020/12/31 11:49:00");
    } catch (ParseException e) {
        e.printStackTrace();
    }

    SystemTime msystemtime = new SystemTime();
    msystemtime.time = dt;
}
```

```
        mHTIManager.SetSystemTime(msystemtime);
    }

    public void Reboot(View view)
    {
        mHTIManager.RebootDevice();
    }

    public void check_autotime()
    {
        boolean autotime = mHTIManager.GetAutoSyncTime();
        m_switch.setChecked(autotime);
    }

    public void Suspend(View view)
    {
        mHTIManager.SuspendDevice();
    }

    public void Resume(View view)
    {
        mHTIManager.ResumeDevice();
    }

    public void CAN_Open(View view)
    {
        CANBUS_BaudRate selectedRate = (CANBUS_BaudRate)
spinnerBaudRate.getSelectedItemAt();

        int re = mHTIManager.CAN_Open("can0", selectedRate);

        if(re!=HTIResult.S_OK.getValue())
```

```
{

    String err = mHTIManager.GetLastError();

    int errcode = mHTIManager.GetLastErrorCode();

    Log.e(TAG, "CAN_Open error (" + errcode + ") = " + err );

    Toast.makeText(MainActivity.this, err, Toast.LENGTH_SHORT).show();

}

}

public void CAN_Close(View view)

{

    int re = mHTIManager.CAN_Close();

    if(re!=HTIResult.S_OK.getValue())

    {

        String err = mHTIManager.GetLastError();

        Log.e(TAG, "CAN_Close error = " + err);

        Toast.makeText(MainActivity.this, err, Toast.LENGTH_SHORT).show();

    }

}

public void CAN_Write(View view)

{

    int[] data = {0xA0, 0xA1, 0xA2, 0xA3, 0xA4, 0xA5, 0xA6, 0xA7};

    int re = mHTIManager.CAN_Write(0x12345678,1,0, data.length,data);

    //re = mHTIManager.CAN_Write(0x12345678,0,0, data.length,data);

    if(re!=HTIResult.S_OK.getValue())

    {

        String err = mHTIManager.GetLastError();
```

```
        Log.e(TAG, "CAN_Write error = " + err);

        Toast.makeText(MainActivity.this, err, Toast.LENGTH_SHORT).show();

    }

}

public void SP_Open(View view)

{

    Log.d(TAG, "[SP_Open]");

    int re = mHTIManager.SerialPort_Open("dev/ttyS1",
UART_baudRate.BaudRate_9600);

    if(re!=HTIResult.S_OK.getValue())

    {

        String err = mHTIManager.GetLastError();

        Log.e(TAG, "SerialPort_Open error = " + err);

        Toast.makeText(MainActivity.this, err, Toast.LENGTH_SHORT).show();

    }

}

public void SP_Close(View view)

{

    int re = mHTIManager.SerialPort_Close("dev/ttyS1");

    if(re!=HTIResult.S_OK.getValue())

    {

        String err = mHTIManager.GetLastError();

        Log.e(TAG, "error = " + err);

        Toast.makeText(MainActivity.this, err, Toast.LENGTH_SHORT).show();

    }

}
```

```
}

public void SP_Write_String(View view)

{

    String msg = editText_test.getText().toString()+"\n";

    int re = mHTIManager.SerialPort_WriteData("dev/ttyS1",msg);

    if(re!=HTIResult.S_OK.getValue())

    {

        String err = mHTIManager.GetLastError();

        Log.e(TAG, "error = " + err);

        Toast.makeText(MainActivity.this, err, Toast.LENGTH_SHORT).show();

    }

}

public void SP_Write(View view)

{

    String msg = editText_test.getText().toString();

    //byte[] temp = toByteArray(msg);

    byte[] ttt = new byte[]

{ (byte)0x41,(byte)0x54,(byte)0x2B,(byte)0x43,(byte)0x47,(byte)0x4D,(byte)0x49,(byte)

0x0A };

    int re = mHTIManager.SerialPort_WriteData("dev/ttyS1",ttt);

    //int re = mHTIManager.SerialPort_WriteData("dev/ttyS1","AT");

    if(re!=HTIResult.S_OK.getValue())

    {

        String err = mHTIManager.GetLastError();

        Log.e(TAG, "error = " + err);

    }

}
```

```
        Toast.makeText(MainActivity.this, err, Toast.LENGTH_SHORT).show();
    }
}

public void SPI_Transfer(View view)
{
    String selected = (String) spinnerSPI.getSelectedItem();

    byte[] data={ (byte)0x01, (byte) 0xaa};

    byte [] out = new byte[data.length];

    int re = mHTIManager.SPI_Transfer(Integer.parseInt(selected),data,out);

    Log.w(TAG, "SPI_Transfer re = " + re);

    if(re!=HTIResult.S_OK.getValue())
    {
        String err = mHTIManager.GetLastError();

        Log.e(TAG, "SPI_Transfer error = " + err);

        Toast.makeText(MainActivity.this, err, Toast.LENGTH_SHORT).show();
    }
    else
    {
        String _out = toHexString(out,out.length);

        Log.i(TAG, "SPI_Transfer : " + _out);
    }
}

public void I2C_Write(View view)
{
    int re = mHTIManager.setI2C(5,0x38 , new int[]{0xAC, 0x33, 0x00});
```

```
Log.w(TAG, "I2C_Write");

if(re!=HTIResult.S_OK.getValue())

{

    String err = mHTIManager.GetLastError();

    Log.e(TAG, "I2C_Write error = " + err);

    Toast.makeText(MainActivity.this, err, Toast.LENGTH_SHORT).show();

}

}

public void I2C_Read(View view)

{

    int [] return_value = new int[6];

    int re = mHTIManager.getI2C(5,0x38 , 6,return_value);

    Log.w(TAG, "I2C_Read");

    if(re!=HTIResult.S_OK.getValue())

    {

        String err = mHTIManager.GetLastError();

        Log.e(TAG, "I2C_Read error = " + err);

        Toast.makeText(MainActivity.this, err, Toast.LENGTH_SHORT).show();

    }

    else

    {

        StringBuilder sb = new StringBuilder();

        for (int v : return_value) {

            sb.append(String.format("0x%02X ", v));

        }

    }

}
```

```
        Log.i(TAG, "[I2C_Read] = " + sb.toString().trim());
    }
}

public void new_GPIO_Write(View view)
{
    String selected = (String) spinnerGPIO.getSelectedItemAt();

    GPIO_Config config = new GPIO_Config();

    int selected_mode = radioGroupGPIO.getCheckedRadioButtonId();

    Log.d(TAG, "selected_mode : " + selected_mode);

    if(selected_mode != -1) {

        RadioButton selectedButton = findViewById(selected_mode);

        String selectedText = selectedButton.getText().toString();

        switch (selectedText)
        {

            case "no IRQ LOW 80":

                config.direction = GPIO_Direction.output;

                config.interrupt = false;

                config.potential = GPIO_Potential.low;

                config.wayToEvok = GPIO_WayToEvok.IRQF_TRIGGER_NONE;

                break;

            case "no IRQ HIGH 90":

                config.direction = GPIO_Direction.output;

                config.interrupt = false;

                config.potential = GPIO_Potential.high;

                config.wayToEvok = GPIO_WayToEvok.IRQF_TRIGGER_NONE;
```

```
        break;

    case "IRQ RISING 51":

        config.direction = GPIO_Direction.input;

        config.interrupt = true;

        config.potential = GPIO_Potential.high;

        config.wayToEvok = GPIO_WayToEvok.IRQF_TRIGGER_RISING;

        break;

    case "IRQ RISING and FALLING 53":

        config.direction = GPIO_Direction.input;

        config.interrupt = true;

        config.potential = GPIO_Potential.high;

        config.wayToEvok =
GPIO_WayToEvok.IRQF_TRIGGER_RISING_and_IRQF_TRIGGER_FALLING;

        break;

    }

}

// 80 no IRQ 1000 0000

GPIO_Config config_80 = new GPIO_Config();

config_80.direction = GPIO_Direction.output;

config_80.interrupt = false;

config_80.potential = GPIO_Potential.low;

config_80.wayToEvok = GPIO_WayToEvok.IRQF_TRIGGER_NONE;

// 51 IRQ 0101 0001

GPIO_Config config_51= new GPIO_Config();

config_51.direction = GPIO_Direction.input;
```

```
config_51.interrupt = true;

config_51.potential = GPIO_Potential.high;

config_51.wayToEvok = GPIO_WayToEvok.IRQF_TRIGGER_RISING;

// 53 IRQ 0101 0011

GPIO_Config config_53= new GPIO_Config();

config_53.direction = GPIO_Direction.input;

config_53.interrupt = true;

config_53.potential = GPIO_Potential.high;

config_53.wayToEvok =
GPIO_WayToEvok.IRQF_TRIGGER_RISING_and_IRQF_TRIGGER_FALLING;

int re = mHTIManager.setGPIO(Integer.parseInt(selected),config);

if(re!=HTIResult.S_OK.getValue())

{

    String err = mHTIManager.GetLastError();

    Log.e(TAG, "new_GPIO_Write error = " + err);

    Toast.makeText(MainActivity.this, err, Toast.LENGTH_SHORT).show();

}

}

public void new_GPIO_Read(View view)

{

    boolean [] value = new boolean[10];

    int re = mHTIManager.getGPIO(value);

    if(re!=HTIResult.S_OK.getValue())

    {
```

```
String err = mHTIManager.GetLastError();

Log.e(TAG, "new_GPIO_Read error = " + err);

Toast.makeText(MainActivity.this, err, Toast.LENGTH_SHORT).show();

}

else

{

    StringBuilder sb = new StringBuilder();

    sb = new StringBuilder();

    for (boolean b : value) {

        sb.append(b ? "1 " : "0 "); // true -> 1, false -> 0

    }

    String hexString = sb.toString().trim();

    Log.i(TAG, "GPIO_Read value: " + hexString);

}

}

public void PWM_Write(View view)

{

    try {

        int pwmChannel =

Integer.parseInt(spinner_pwm.getSelectedItem().toString());

        int enable = checkBox_pwm.isChecked() ? 1 : 0;

        String periodStr = editText_pwm_period.getText().toString();

        String dutyStr = editText_pwm_duty.getText().toString();

        int period = Integer.parseInt(periodStr);

        int duty = Integer.parseInt(dutyStr);
```

```
int ret = mHTIManager.setPWM(pwmChannel, enable, period, duty);

Log.i(TAG, "PWM set result: " + ret);

} catch (NumberFormatException e) {

    Log.e(TAG, "NumberFormatException");

    Toast.makeText(this,

        "NumberFormatException",

        Toast.LENGTH_SHORT).show();

    }

}

}
```